Project 3 – Maze Runner!

Due: 3/24/2021 11:59:59 pm

Goal. In this project, you will create an efficient solver for two-dimensional mazes. Each maze consists of a grid of positions. Someone moving through the maze can generally move in one of the four compass directions (North, South, East, West) to advance to an adjacent position; however, walls may block some of these possibilities. There is also a start position, and an exit position from the maze. For a given maze, starting from the start position, your program should either return a solution (i.e. a path to the exit) or determine that no solution exists. ***Your program should also run faster than all our single-threaded solvers*** when run on a multi-core machine. The implementations details for how to solve the maze are left up to you, subject to certain restrictions given below, and **you may use any default Java library classes you wish**. This includes synchronizers, executors, and fork/join pools.

Contest. This project will also include a contest component by which you may earn some extra credit, depending on the execution speed of your solution. In the contest, your program will be run against programs submitted by other students in the class on a contest server; the results will be posted to a shared leaderboard. This board will detail every submission's running time on various undisclosed test mazes. Extra credit will be awarded based on the performance of your program in comparison with others**. Submission to the submit server will automatically queue your program to run on the contest server**, once the contest server is activated. Further details regarding the contest server will be posted to Piazza.

Maze Restrictions. For the purposes of this project you may assume the following about the mazes your code will be run on.

- Mazes will contain *at most* one solution.
- Mazes will NOT contain any cycles or loops.
- Mazes will fit into a grid measuring 20,000 cells by 20,000 cells.
- Mazes will never be edge cases such as 0x0 or 1x1

Mazes in P3 contain single-sided walls. Single-sided walls are special walls that only allow one-way movement (i.e. cell A can go to cell B, but cell B cannot go cell A). As a result, **meet-in-the-middle and backtracking approaches will NOT work.** Consider instead simply conducting your search from the start of the maze to the end.

Project skeleton code. The code we will provide you contains the following class files. You may make changes to the following.

- `StudentMTMazeSolver.java`. This class will contain your solution, and it is the one we will test. It extends the class `MazeSolver`. *You must ensure that your class is a subclass of the* `MazeSolver` *class!* Here are more details about your class.

    - Constructor `StudentMTMazeSolver()`. The constructor takes a `Maze` object to solve and stores it for later reference.
    - `List<Direction> solve()`. This method computes and returns a solution to the maze object stored in the instance field, if one exists. The solution should consist of a list of directions taken at every cell beginning from the start and leading to the endpoint. If no solution exists, this method should return null.
- `Main.java`. This class is intended to facilitate testing your code and can be modified as desired. It will not be run by the submit server, and any changes you make to it will be overwritten on the contest server, so **please ensure that there is nothing in this file that the rest of your program depends on**. Specific methods in this class include the following.
    - `read()`. This method reads a `Maze` object from a file and stores it in private instance variable `maze`.
    - `solve()`. This method executes the solvers indicated and prints out the results.
    - `initDisplay()`. This method displays a graphical representation of a maze for debugging purposes. Your maze solver can also be configured to draw its path on the display to make tracing easier.
    - `main()`. This method runs opens a `Maze` from a file, runs the given solvers, and displays the results alongside a picture of the `Maze` if requested.

The following files are also provided, and you may make use of them. ***You should not make any modifications to these files**, however!* Our testing will overwrite these files in your submission with the original ones from the skeleton.

- `Maze.java`. Represents a two-dimensional maze as an `AtomicIntegerArray` and provides methods returning information about the maze. The method `checkSolution()` can be used to verify results. For this project all mazes are read and deserialized from files.
- `Position.java`. Represents a cell in the `Maze`.
- `Direction.java`. Represents the four compass directions, used to refer to neighboring `Positions`.
- `Move.java`. Represents a move by storing a `Position` along with the `Direction` of the move. Also has a reference to the previous `Move` for traversal-building.
- `Choice.java`. Represents movement possibilities for a `Position` in a `Maze` as part of a traversal. In addition to `Position`, it stores the `Direction` of the previous `Position` as well as valid `Directions` to proceed in. A wall at a given `Position` is represented by a lack of `Choice` in a given `Direction`.
- `MazeSolver.java`. Superclass of all maze solvers. Whatever your implementation is, it MUST subclass this class.
- `SkippingMazeSolver.java`. `MazeSolver` that for each move, follows a `Direction` and progresses through the `Maze` until a `Choice` is reached, allowing for a more concise representation of a traversal. This partial `Direction` list can be

converted to a full `Direction` list upon returning. A method to color `Positions` may be used in debugging.

In addition, we have included several fully-implemented single-threaded solvers – `STMazeSolverBFS.java`, `STMazeSolverDFS.java` and `STMazeSolverRec.java` – for you to compare times against and/or use as a basis for your solution.

Your implementation. You should implement your solution in the file `StudentMTMazeSolver.java`, which must be a subclass of `MazeSolver`. You may also modify `Main.java`, if you wish, and add additional class files beyond those in the skeleton code if this will help with your solution. Please do not modify any of the other skeleton files beyond `StudentMTMazeSolver.java` and `Main.java`. You should also ensure that nothing in your solution depends on anything in `Main.java`.

Using code from the BFS and DFS maze solvers can be a good starting point for implementing your own solution. Be sure to take a look at those and think about how threads can be used to improve those solutions. Your solution will not be penalized for copying code from these files.

If your solution depends on the number of available processors on the multicore machine at runtime, consider using *Runtime.getRuntime().availableProcessors()* to fetch this value.

Testing. Among the tests we will use for this project are races where we will compare your solver's time against a single-threaded solver's time. For full credit, your program should be able to consistently win these races. We will, of course, also test your output for correctness.

For offline testing, we have provided several maze files for you to run, as well as some single-threaded solvers you can use as benchmarks. Maze files ending with 'u' do not have a solution (they are **u**nsolvable). When testing larger maze files, you may need to comment out the recursive maze solver in `Main.java` to avoid stack overflows from recursive calls (this is expected behavior, do not worry). As long as your solution is consistently faster than the DFS, BFS, and recursive maze solvers that have been provided, you should be in good shape to pass the tests on the submit server.

Because of the provided files and the presence of the contest server, we will NOT have public tests on the submit server. **Discussion for this project is encouraged**, although as usual you are not allowed to share code with your colleagues.

Submission. Submit a .zip file containing your project files to the CS submit (submit.cs.umd.edu). You may submit an unlimited number of times. ***Please do not include any maze files in your project submission!*** Failure to adhere to this request may hinder your program's ability to run.

Grading. Project 3 is 100% secret tests. There are a total of 8 secret tests (some worth 12 points, others worth 13 points) adding up to 100 points total.

DeepThought2. You may use the DeepThought2 cluster to run and test project 3. Use of DeepThought2 is **optional.** To run your code on the cluster, follow the instructions below:

The steps below will be performed on your local machine:

1) Export your project code into a zip file, just like you normally would when uploading your project to the submit server. Also, obtain a copy of the maze-distribution zip file
2) Open up a terminal/command prompt on your local machine and enter the following command to copy your project code and the maze distribution files over to deepthought2:
    a. scp <path to project zip> <path to maze-dist zip> <username>@login.deepthought2.umd.edu:~/.
3) Once you are prompted, enter your password and the zip files will be copied over to deepthought2.
4) From your local terminal/command prompt, login to your deepthought2 account with the following command:
    a. ssh <username>@login.deepthought2.umd.edu.
5) Enter your password and you will arrive at a tcsh shell.

The steps below will be performed on the DeepThought2 cluster

1) If you type `*java -version*` you will see that an older version of java is being used on the cluster. To remedy this, enter the `*module load java/8*` command. Type in `*java -version*` again and you should see java 1.8 as the version number. If you don't want to type in the module load command every time you login to the cluster, add `*module load java/8*` to your *~/.cshrc.mine* config file.
2) Unzip your project code and the maze distribution files by entering `*unzip <project zip>*` and `*unzip <maze-dist zip>*`. If you type `*ls*`, you should now see a **maze-dist** and **p3** directory.
3) You will need to update the path to your maze distribution file in Main.java so obtain the new full path to the maze-dist directory by entering the following command (assuming you are still in the same directory that the maze-dist directory is in): `*readlink -f maze-dist*`.
4) Copy the output of this command and update the variable that stores the path to the maze-dist directory in Main.java by using a text editor such nano or vim. The relative path to the Main.java file is *"p3/src/cmsc433/p3/Main.java"*. Remember to add a forward slash to the end of the string path variable so it ends up looking like *"/path/to/my/maze-dist/"*.
5) Now, you can compile your code. Enter the **p3** directory by typing `*cd p3*` and then type `*ls*` to ensure that you have a **src** and **bin** directory. Then, enter the following command to compile your code:
    a. `javac -d bin/ src/cmsc433/p3/*.java`
6) If your code compiles without any errors, enter the **bin** directory (assuming you are still in the p3 directory) by entering `*cd bin*`. Then, enter the following to run your program

a. *`java cmsc433/p3/Main`*

Once you confirm that these steps work, place the command(s) for running your code into a script and schedule it as job for the cluster to run by following the instructions found here: http://hpcc.umd.edu/hpcc/help/jobs.html